CS 229, Summer 2025
Problem Set #2

Baran Peters (baranp@stanford.edu)

Due Friday, July 25 at 11:59 pm on Gradescope.

Notes: (1) These questions require thought, but do not require long answers. Please be as concise as possible.

(2) If you have a question about this homework, we encourage you to post your question on our Ed forum, at `https://edstem.org/us/courses/79825/discussion`.

(3) If you missed the first lecture or are unfamiliar with the collaboration or honor code policy, please read the policy on the course website before starting work.

(4) For the coding problems, you may not use any libraries except those defined in the provided `environment.yml` file. In particular, ML-specific libraries such as scikit-learn are not permitted.

(5) The due date is Friday, July 25 at 11:59 pm. If you submit after Friday, July 25 at 11:59 pm, you will begin consuming your late days. The late day policy can be found in the course website: Course Logistics and FAQ.

All students must submit an electronic PDF version of the written question including plots generated from the codes. We **require you to typeset your solutions via LaTeX**. All students must also submit a zip file of their source code to Gradescope, which should be created using the `make_zip.py` script. You should make sure to (1) restrict yourself to only using libraries included in the `environment.yml` file, and (2) make sure your code runs without errors. Your submission may be evaluated by the auto-grader using a private test set, or used for verifying the outputs reported in the writeup. Please make sure that your PDF file and zip file are submitted to the corresponding Gradescope assignments respectively. We reserve the right to not give any points to the written solutions if the associated code is not submitted.

Honor code: We strongly encourage students to form study groups. Students may discuss and work on homework problems in groups. However, each student must write down the solutions independently, and without referring to written notes from the joint session. In other words, each student must understand the solution well enough in order to reconstruct it by themself. In addition, each student should write on the problem set the set of people with whom s/he collaborated. Further, because we occasionally reuse problem set questions from previous years, we expect students not to copy, refer to, or look at the solutions in preparing their answers. It is an honor code violation to intentionally refer to a previous year's solutions.

Regarding Notation: The notation used in this problem set matches the notation used in the lecture notes. Some notable differences from lecture notation:

- The superscript "(i)" represents an index into the training set – for example, $x^{(i)}$ is the i-th feature vector and $y^{(i)}$ is the i-th output variable. In lecture notation, these would instead be expressed as x_i and y_i.
- The subscript j represents an index in a vector – in particular, $x_j^{(i)}$ represents the j-th feature in the feature vector $x^{(i)}$. In lecture notation, $x_j^{(i)}$ would be $h_j(x_i)$ or $x_i[j]$.
- The vector that contains the weights parameterizing a linear regression is expressed by the variable θ, whereas lectures use the variable $\mathbf{w}$. As such, $\theta_0 = w_0$, $\theta_1 = w_1$, and so on.

An overview of this notation is also given at the beginning of the lecture notes.

1. **[25 points] Poisson Regression**

In this question we will construct another kind of a commonly used GLM, which is called Poisson Regression. In a GLM, the choice of the exponential family distribution is based on the kind of problem at hand. If we are solving a classification problem, then we use an exponential family distribution with support over discrete classes (such as Bernoulli, or Categorical). Similarly, if the output is real valued, we can use Gaussian or Laplace (both are in the exponential family). Sometimes the desired output is to predict counts, for example, predicting the number of emails expected in a day, or the number of customers expected to enter a store in the next hour, etc. based on input features (also called covariates). You may recall that a probability distribution with support over integers (i.e., counts) is the Poisson distribution, and it also happens to be in the exponential family.

In the following sub-problems, we will start by showing that the Poisson distribution is in the exponential family, derive the functional form of the hypothesis, derive the update rules for training models, and finally using the provided dataset to train a real model and make predictions on the test set. Please refer to the lecture notes if you would like a refresher on any of the terms or notations covered in this question.

(a) [5 points] Consider the Poisson distribution parameterized by λ:

$$p(y;\lambda) = \frac{e^{-\lambda}\lambda^y}{y!}.$$

(Here y has positive integer values and $y!$ is the factorial of y.) Show that the Poisson distribution is in the exponential family, and clearly state the values for $b(y)$, η, $T(y)$, and $a(\eta)$.

Answer:

i. $p(y;\lambda) = \frac{e^{-\lambda}\lambda^y}{y!} \rightarrow$ PDF of Poisson Distribution

To prove i) is part of the exponential family, we need to prove it follows this form:

$$p(y;\eta) = b(y)\exp(\eta^T T(y) - a(\eta))$$

With $\lambda^y = e^{\log(\lambda^y)} = e^{y\log(\lambda)}$, we can say:

$$\begin{aligned} p(y;\lambda) &= \frac{1}{y!}\exp(y\log(\lambda) - \lambda) \\ &= \underbrace{\frac{1}{y!}}_{b(y)} \exp(\underbrace{y}_{T(y)}\underbrace{\log(\lambda)}_{\eta^T} - \underbrace{\lambda}_{a(\eta)}) \end{aligned}$$

As $a(\eta) = \lambda$ is not a function of η, we can rewrite it as $a(\eta) = e^{\eta}$
(with $\log(\lambda) = \eta \rightarrow \lambda = e^{\eta}$)

So we have:

$$\begin{aligned} b(y) &= \frac{1}{y!} \\ T(y) &= y \\ \eta &= \log(\lambda) \\ a(\eta) &= e^{\eta} \end{aligned}$$

(b) [3 points] Consider performing regression using a GLM model with a Poisson response variable. What is the canonical response function for the family? (You may use the fact that a Poisson random variable with parameter λ has mean λ.)

Answer:

The canonical response function for GLM models is defined as follows:

$$g(\eta) = E[T(y); \eta] \quad \text{(assuming } \eta = \theta^T x \text{ and } T(y) = y)$$

We know based on the problem's hint that $E[y] = \lambda$, and from a) that $\lambda = e^{\eta}$, so the canonical response function for models with a Poission response variable are as follows:

$$g(\eta) = \lambda = e^{\eta}$$

(c) [7 points] For a training set $\{(x^{(i)}, y^{(i)}); i = 1, \ldots, n\}$, let the log-likelihood of an example be $\log p(y^{(i)}|x^{(i)}; \theta)$. By taking the derivative of the log-likelihood with respect to θ_j, derive the stochastic gradient ascent update rule for learning using a GLM model with Poisson responses y and the canonical response function.

Answer:

Based on GLM and the results in a), we have:

$$p(y|x;\theta) = \frac{1}{y!}\exp(\theta^T x y - e^{\theta^T x})$$

The derivative of the log likelihood is defined as follows:

$$\begin{aligned}
\frac{\partial}{\partial\theta_j}\log p(y|x;\theta) &= \frac{\partial}{\partial\theta_j}(\log(\frac{1}{y!}) + (\theta^T x)y - e^{\theta^T x}) \quad \text{(removing irrelevant terms)} \\
&= \frac{\partial}{\partial\theta_j}\left(y\sum_k \theta_k x_k - e^{\sum_k \theta_k x_k}\right) \\
&= y x_j - e^{\theta^\top x} x_j \\
&= x_j\left(y - e^{\theta^\top x}\right)
\end{aligned}$$

We want to maximize this log likelihood given some theta - via SGD and the gradient $\frac{\partial}{\partial\theta_j}\log p(y|x;\theta)$ we can nudge on θ_j in the right direction.

$$\theta_j := \theta_j + \alpha\frac{\partial}{\partial\theta_j}l(\theta)$$
$$\theta_j := \theta_j + \alpha(y - e^{\theta^T x})x_j$$

(d) [10 points] **Coding problem**

Consider a website that wants to predict its daily traffic. The website owners have collected a dataset of past traffic to their website, along with some features which they think are useful in predicting the number of visitors per day. The dataset is split into train/valid sets and the starter code is provided in the following files:

- `src/poisson/{train,valid}.csv`

- `src/poisson/poisson.py`

We will apply Poisson regression to model the number of visitors per day. Note that applying Poisson regression in particular assumes that the data follows a Poisson distribution whose natural parameter is a linear combination of the input features (*i.e.*, $\eta = \theta^T x$). In `src/poisson/poisson.py`, implement Poisson regression for this dataset and use *full batch gradient ascent* to maximize the log-likelihood of θ. For the stopping criterion, check if the change in parameters has a norm smaller than a small value such as 10^{-5}.

Using the trained model, predict the expected counts for the **validation set**, and create a scatter plot between the true counts vs predicted counts (on the validation set). In the scatter plot, let x-axis be the true count and y-axis be the corresponding predicted expected count. Note that the true counts are integers while the expected counts are generally real values.

Answer:

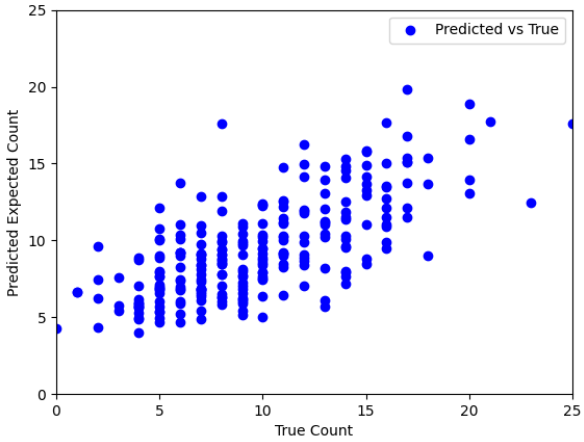


Figure 1: Poisson True vs. Predicted Values

2. **[15 points] Convexity of Generalized Linear Models**

In this question we will explore and show some nice properties of Generalized Linear Models, specifically those related to its use of Exponential Family distributions to model the output.

Most commonly, GLMs are trained by using the negative log-likelihood (NLL) as the loss function. This is mathematically equivalent to Maximum Likelihood Estimation (*i.e.,* maximizing the log-likelihood is equivalent to minimizing the negative log-likelihood). In this problem, our goal is to show that the NLL loss of a GLM is a *convex* function w.r.t the model parameters. As a reminder, this is convenient because a convex function is one for which any local minimum is also a global minimum, and there is extensive research on how to optimize various types of convex functions efficiently with various algorithms such as gradient descent or stochastic gradient descent.

To recap, an exponential family distribution is one whose probability density can be represented as

$$p(y;\eta) = b(y)\exp(\eta^T T(y) - a(\eta)),$$

where η is the *natural parameter* of the distribution. Moreover, in a Generalized Linear Model, η is modeled as $\theta^T x$, where $x \in \mathbb{R}^d$ are the input features of the example, and $\theta \in \mathbb{R}^d$ are learnable parameters. In order to show that the NLL loss is convex for GLMs, we break down the process into sub-parts, and approach them one at a time. Our approach is to show that the second derivative (*i.e.,* Hessian) of the loss w.r.t the model parameters is Positive Semi-Definite (PSD) at all values of the model parameters. We will also show some nice properties of Exponential Family distributions as intermediate steps.

For the sake of convenience we restrict ourselves to the case where η is a scalar. Assume $p(Y|X;\theta) \sim \text{ExponentialFamily}(\eta)$, where $\eta \in \mathbb{R}$ is a scalar, and $T(y) = y$. This makes the exponential family representation take the form

$$p(y;\eta) = b(y)\exp(\eta y - a(\eta)).$$

Note that the above probability density is for a single example (x, y).

(a) [5 points] Derive an expression for the mean of the distribution. Show that $\mathbb{E}[Y;\eta] = \frac{\partial}{\partial \eta} a(\eta)$ (note that $\mathbb{E}[Y;\eta] = \mathbb{E}[Y|X;\theta]$ since $\eta = \theta^T x$). In other words, show that the mean of an exponential family distribution is the first derivative of the log-partition function with respect to the natural parameter.

Hint: Start with observing that $\frac{\partial}{\partial \eta}\int p(y;\eta)dy = \int \frac{\partial}{\partial \eta} p(y;\eta)dy$.

Answer:

Given:

1. $p(y|x;\theta) \sim \text{ExponentialFamily}(\eta)$ where $\eta \in \mathbb{R}$ is a scalar and $T(y) = y$
2. $p(y,\eta) = b(y)\exp(\eta y - a(\eta))$

The integral of a PDF equals always 1:

$$\int p(y,\eta)dy = 1$$

Let's differentiate both sides by η:

i. $\frac{\partial}{\partial\eta}\int p(y,\eta)dy = \frac{\partial}{\partial\eta}1 = 0$
$\Rightarrow \int \frac{\partial}{\partial\eta}p(y,\eta)dy = 0$
The derivative $\frac{\partial p(y,\eta)}{\partial\eta}$ is defined as:

$$\begin{aligned}\frac{\partial p(y,\eta)}{\partial\eta} &= \frac{\partial}{\partial\eta}(b(y)\exp(\eta y - a(\eta))) = (y - a'(\eta))b(y)\exp(\eta y - a(\eta)) \\ &= (y - a'(\eta))p(y,\eta)\end{aligned}$$

Inserting into i)

ii. $\int (y - a'(\eta))p(y,\eta)dy = 0$

$$\underbrace{\int y \cdot p(y,\eta)dy}_{\text{by definition } E[y,\eta]} - a'(\eta) \underbrace{\int p(y,\eta)dy}_{\substack{\text{we know by definition} \\ \text{that this equals 1}}} = 0$$

$E[y,\eta] - a'(\eta) = 0 \Rightarrow E[y,\eta] = a'(\eta)$

(b) [5 points] Next, derive an expression for the variance of the distribution. In particular, show that $\text{Var}(Y;\eta) = \frac{\partial^2}{\partial\eta^2}a(\eta)$ (again, note that $\text{Var}(Y;\eta) = \text{Var}(Y|X;\theta)$). In other words, show that the variance of an exponential family distribution is the second derivative of the log-partition function w.r.t. the natural parameter.

Hint: Building upon the result in the previous sub-problem can simplify the derivation.

Answer:

The variance of a random variable is defined as:

$$\text{Var}(y) = E[(y - E[y])^2] = E[y^2] - E[y]^2$$

In our case:

$$\text{Var}(y,\eta) = E[y^2;\eta] - E[y;\eta]^2$$

From a), we know that $E[y;\eta] = \frac{\partial a(\eta)}{\partial\eta}$. Let's differentiate again by η:

$$\begin{aligned}\text{iii)}\ \frac{\partial}{\partial\eta}E[Y|\eta] &= \frac{\partial^2}{\partial\eta^2}a(\eta) \\ \text{c=)}\ \frac{\partial}{\partial\eta}\int y \cdot p(y|\eta)dy &= \frac{\partial^2}{\partial\eta^2}a(\eta) \\ &= \int y \cdot \frac{\partial}{\partial\eta}p(y|\eta)dy \quad \text{(insert from a))} \\ &= \int y \cdot (y - a'(\eta))p(y|\eta)dy \\ &= \underbrace{\int y^2 p(y|\eta)dy}_{E[Y^2|\eta]} - a'(\eta)\underbrace{\int y\, p(y|\eta)dy}_{E[Y|\eta]} \\ &= E[Y^2;\eta] - a'(\eta) \cdot E[Y;\eta] \\ &= E[Y^2;\eta] - (E[Y;\eta])^2 = \frac{\partial^2}{\partial\eta^2}a(\eta)\end{aligned}$$

→ That's the definition of the Variance Var(y, η).

(c) [5 points] Finally, write out the loss function $\ell(\theta)$, the NLL of the distribution, as a function of θ. Then, calculate the Hessian of the loss w.r.t θ, and show that it is always PSD. This concludes the proof that NLL loss of GLM is convex.

Hint 1: Use the chain rule of calculus along with the results of the previous parts to simplify your derivations.

Hint 2: Recall that variance of any probability distribution is non-negative.

Answer:

c)

The NLL of the distribution is:

$$\begin{aligned} l(\theta) &= -\log(p(y|x)) \\ &= -\log(b(y)\exp(\eta y - a(\eta))) \quad \text{where y and x are given and } \eta = \theta^T x \\ &= -\log b(y) - \eta y + a(\eta) \end{aligned}$$

Gradient $\nabla l(\theta)$:

Based on the chain rule $\frac{dl}{d\theta} = \frac{dl}{d\eta}\frac{d\eta}{d\theta}$

$$\begin{aligned} \frac{\partial l}{\partial \eta} &= a'(\eta) - y \\ \frac{\partial \eta}{\partial \theta} &= x \\ \rightarrow \text{So } \frac{\partial l}{\partial \theta} &= (a'(\eta) - y) \cdot x \end{aligned}$$

Hessian $H = \nabla^2 l(\theta)$:

Based on the chain rule: $\frac{\partial^2 l}{\partial \theta^2} = \frac{\partial}{\partial \theta}(\underbrace{(a'(\eta) - y)}_{z} x) = \frac{\partial \nabla l(\theta)}{\partial z}\frac{\partial z}{\partial \eta}\frac{\partial \eta}{\partial \theta}$

$$\begin{aligned} \frac{\partial \nabla l(\theta)}{\partial z} &= x \\ \frac{\partial z}{\partial \eta} &= a''(\eta) \\ \frac{\partial \eta}{\partial \theta} &= x \end{aligned}$$

→ So $\frac{\partial^2 l}{\partial \theta^2} = x(a''(\eta) \cdot x^T) = a''(\eta) \cdot xx^T$ → Outer Product scaled by $a''(\eta)$

Show that H is PSD via Quadratic Form:

$z^T H z \geq 0$ for all z, then H is PSD

$$z^T H z \geq 0 = z^T (a''(\eta) x x^T) z = a''(\eta) z^T (x x^T) z = \underbrace{a''(\eta)}_{\substack{\text{As shown in b),} \\ \text{this is } Var(y|\eta), \\ \text{which is always non-negative}}} \quad \underbrace{(z^T x)(x^T z)}_{\text{square of } z^T x \rightarrow \text{always } \geq 0}$$

$\rightarrow z^T H z \geq 0 \rightarrow$ The Hessian of the NLL is always PSD, and hence it must be convex.

Remark: The main takeaways from this problem are:

- Any GLM model is convex in its model parameters.
- The exponential family of probability distributions are mathematically nice. Whereas calculating mean and variance of distributions in general involves integrals (hard), surprisingly we can calculate them using derivatives (easy) for exponential family.

3. **[25 points] Logistic Regression: Training stability**

In this problem, we will be delving deeper into the workings of logistic regression. The goal of this problem is to help you develop your skills debugging machine learning algorithms (which can be very different from debugging software in general).

(a) [10 points] **Coding question**

In lecture we saw the average empirical loss for logistic regression:

$$J(\theta) = -\frac{1}{n}\sum_{i=1}^{n}\left(y^{(i)}\log(h_\theta(x^{(i)})) + (1-y^{(i)})\log(1-h_\theta(x^{(i)}))\right),$$

where $y^{(i)} \in \{0,1\}$, $h_\theta(x) = g(\theta^T x)$ and $g(z) = 1/(1+e^{-z})$.

Implement logistic regression using gradient descent in `src/logreg_stability/logreg.py`. Starting with $\theta = \vec{0}$, run gradient descent until the updates to θ are small: Specifically, train until the first iteration k such that $\|\theta_k - \theta_{k-1}\|_1 < \epsilon$, where $\epsilon = 1 \times 10^{-5}$, or for a maximum of 10^5 iterations. Then, perform logistic regression on dataset A in `src/logreg_stability/ds1_a.csv`. You can run the code by simply executing `python logreg.py` in the `src/logreg_stability` directory.

Include a plot of the training data with x_1 on the horizontal axis and x_2 on the vertical axis. To visualize the two classes, use a different symbol for examples $x^{(i)}$ with $y^{(i)} = 0$ than for those with $y^{(i)} = 1$. On the same figure, plot the decision boundary found by logistic regression (i.e., line corresponding to $p(y|x) = 0.5$).

Note: If you want to print the loss during training, you may encounter some numerical instability issues. Recall that the loss function on an example (x, y) is defined as

$$y\log(h_\theta(x)) + (1-y)\log(1-h_\theta(x)),$$

where $h_\theta(x) = (1+\exp(-x^\top\theta))^{-1}$. Technically speaking, $h_\theta(x) \in (0,1)$ for any $\theta, x \in \mathbb{R}^d$. However, in Python a real number only has finite precision. So it is possible that in your implementation, $h_\theta(x) = 0$ or $h_\theta(x) = 1$, which makes the loss function ill-defined. A typical solution to the numerical instability issue is to add a small perturbation. In this case, you can compute the loss function using

$$y\log(h_\theta(x)+\epsilon) + (1-y)\log(1-h_\theta(x)+\epsilon),$$

instead, where ϵ is a very small perturbation (for example, $\epsilon = 10^{-5}$).

Answer:

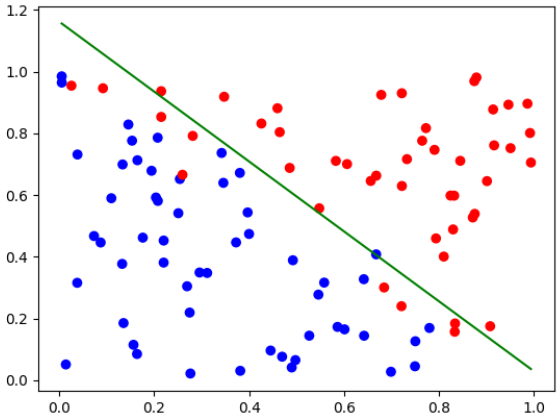


Figure 2: LR decision boundary dataset A

(b) [7 points] Perform logistic regression on dataset B in `src/logreg_stability/ds1_b.csv`, and include the same plot as for dataset A in the previous subquestion. What is the most notable difference in training the logistic regression model on datasets A and B? Investigate why the training procedure behaves unexpectedly on dataset B, but not on A. Provide hard evidence (in the form of math, code, plots, etc.) to corroborate your hypothesis for the misbehavior. Remember, you should address why your explanation does *not* apply to A.

Hint: The issue is not a numerical rounding or over/underflow error.

Answer:

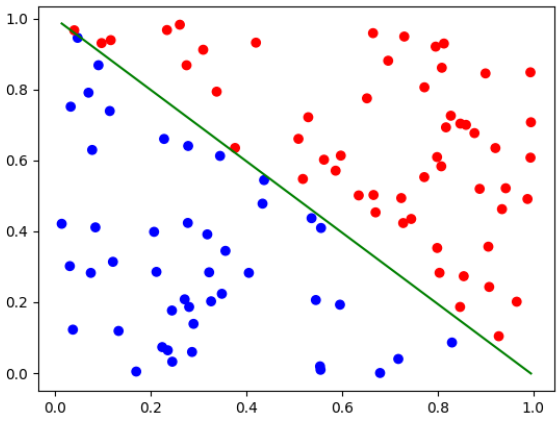


Figure 3: LR decision boundary for dataset B

A:

- Seems to oscillate between two points while converging
  - Change in parameters θ from iter > 5000 (constant (roughly)), while the loss stays constant
- Loss stays relatively high (because points not linearly separable)

B:

- Quickly converges to a very small loss $J(\theta) < 2$, whereas
- Model parameters seem to grow continuously, steadily, while loss also sinks – but both slowly in the range of 10^{-3} per iteration.

A vs. B:

A seems to be inseparable in comparison to B, where you can draw a clear cut.

That is the reason why B's parameters explode – the model gets too confident! The only way to lower the loss $J(\theta)$, is to maximize the term $\log(h_\theta(x))$, or minimize in case of $y = 1$ $\log(1 - h_\theta(x))$. This is why the parameters take high values! As the exponential function in $h_\theta(x)$ never reaches 0 and is strictly increasing/decreasing, the loss will sink continuously

This is not the case with dataset A, as the loss has, due to its inseparability, a linearwise asymptote, and starts to oscillate around that median between the two configurations of θ.

The logs at logs_a.txt and logs_b.txt are evidence of my findings.

(c) [3 points] For each of these possible modifications, state whether or not it would lead to the provided training algorithm converging on datasets such as B. Justify your answers.

i. Using a different constant learning rate.

ii. Decreasing the learning rate over time (e.g. scaling the initial learning rate by $1/t^2$, where t is the number of gradient descent iterations thus far).

iii. Linear scaling of the input features.

iv. Adding a regularization term $\|\theta\|_2^2$ to the loss function.

v. Adding zero-mean Gaussian noise to the training data or labels.

Answer:

i. The learning rate will not ultimately effect whether we would converge, as it would either lead to bigger or smaller gradient updates but not lead to a finite minimum in the loss landscape.

ii. Same reasoning as above, the learning rate doesn't affect whether we have a finite minimum to converge to. It might help us practically to "converge" because the parameter updates will be small enough so that our learning algorithm will stop (if difference of parameters $< \epsilon$).

iii. If we scale our parameters by e.g. $1/n$, our training algorithm will just need b times as long to reach a specific magnitude of parameters, and this will not influence the loss landscape and having a convergable minimum.

iv. A regularization term $\|\|\|\theta\|\|\|^2$ would most definitely help our learning algorithm to converge, as the penalty induced would grow the bigger, the more advanced our convergence process is. There will be a certain point, when the regularization term and fit loss will balance out when we reached a finite minimum.

v. This might cause the data points/labels to spread out in a way that makes it linearly separable and our training algorithm to converge to a finite minimum, but it might cause the opposite based on noise being inherently random.

(d) [5 points] **Coding question** To attempt to fix the issues brought up in the previous subquestions, implement L2 regularization in `src/logreg_stability/logreg.py`. Specifically, modify the gradient update to reflect the following loss function:

$$J(\theta) = -\frac{1}{n}\sum_{i=1}^{n}\left(y^{(i)}\log(h_\theta(x^{(i)})) + (1-y^{(i)})\log(1-h_\theta(x^{(i)}))\right) + \frac{1}{2}\lambda\|\theta\|_2^2. \tag{1}$$

Run the same code as in the previous subquestion using $\lambda = 0.01$. For both datasets, print the final value of θ, and compare it to the final value of θ without regularization. Additionally, include a plot of the decision boundary and training data for both datasets and comment on the results qualitatively. How does the decision boundary compare to the one obtained without regularization, especially with respect to the worst misclassified example; in what cases may this be desirable?

Answer:

Dataset	**Final θ (with regularization)**
A	$[-85.22,\ \ 83.00,\ \ 73.40]$
B	$[-265.60,\ \ 267.25,\ \ 265.40]$

Table 1: Final model parameters (θ) after training with regularization

Dataset B seems to have smaller parameters, having decreased for each component, being now in the range of Dataset A's parameters. The model's decision boundary has shifted a bit lower, and now isn't perfectly classifying the data anymore; the rate of false positives has increased.

$\rightarrow$ Bias has increased, while the variance decreased.

Dataset A seems to have a similar phenomenon where the decision boundary is not as close anymore to the "boundary zone" where points of different label are frequent, but shifts towards the red points, increasing false negatives. The flexibility of the model seems to decrease while the bias increases.

What we observe with A & B would be desirable if we tried to train a model that should generalize across different datasets, which in themselves have considerable noise. If we assume that the data we are training on is not noisy, we could easily make the model a bit more flexible and fit the data a bit more for better performance.

Regularization also provides benefits besides that as well, such as sparsity, which leads to higher understandability, as it forces the model implicitly to prioritize *relevant* features and keeps weights in a stable range where numerical errors are also reduced.

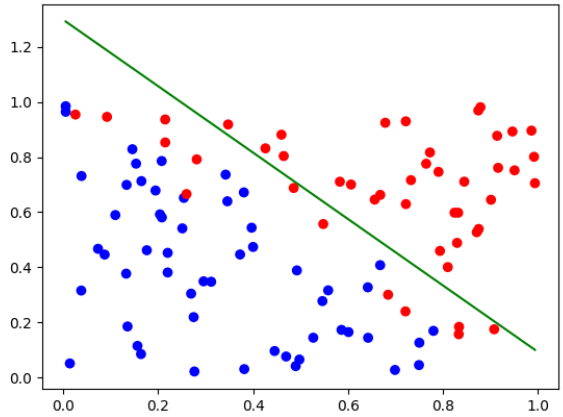


Figure 4: LR decision boundary for dataset A (Regularized)

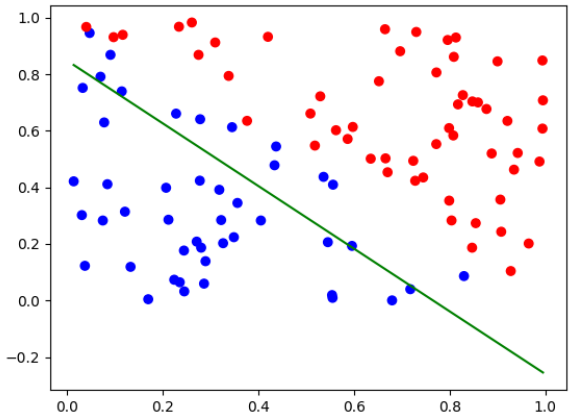


Figure 5: LR decision boundary for dataset B (Regularized)

4. **[25 points] Learning Imbalanced dataset**

In this problem, we study how to learn a classifier from an imbalanced dataset, where the marginal distribution of the classes/labels are imbalanced. Imbalanced datasets are ubiquitous in real-world applications. For example, in the spam detection problem, the training dataset usually has only a small fraction of spam emails (positive examples) but a large fraction of ordinary emails (negative examples). For simplicity, we consider binary classification problem where the labels are in $\{0, 1\}$ and the number of positive examples is much smaller than the number of negative examples.

(a) [10 points] **Evaluation metrics**

In this sub-question, we discuss the common evaluation metrics for imbalanced dataset. Suppose we have a validation dataset and for some $\rho \in (0, 1)$, we assume that ρ fraction of the validation examples are positive examples (with label 1), and $1 - \rho$ fraction of them are negative examples (with label 0).

Define the accuracy as

$$A \triangleq \frac{\#\text{examples that are predicted correctly by the classifier}}{\#\text{examples}}$$

(i) (3 point) Show that for any dataset with ρ fraction of positive examples and $1 - \rho$ fraction of negative examples, there exists a (trivial) classifier with accuracy at least $1 - \rho$.

The statement above suggests that the accuracy is not an ideal evaluation metric when ρ is close to 0. E.g., imagine that for spam detection ρ can be smaller than 1%. The statement suggests there is a trivial classifier that gets more than 99% accuracy. This could be misleading — 99% seems to be almost perfect, but actually you don't need to learn anything from the dataset to achieve it.

Therefore, for imbalanced dataset, we need more informative evaluation metrics. We define the number of true positive, true negative, false positive, false negative examples as

$$TP \triangleq \#\text{ positive examples with a correct (positive) prediction}$$
$$TN \triangleq \#\text{ negative examples with a correct (negative) prediction}$$
$$FP \triangleq \#\text{ negative examples with a incorrect (positive) prediction}$$
$$FN \triangleq \#\text{ positive examples with a incorrect (negative) prediction}$$

Define the accuracy of positive examples as

$$A_1 \triangleq \frac{TP}{TP + FN} = \frac{\#\text{ positive examples with a correct (positive) prediction}}{\#\text{ positive examples}}$$

Define the accuracy of negative examples as

$$A_0 \triangleq \frac{TN}{TN + FP} = \frac{\#\text{ negative examples with a correct (negative) prediction}}{\#\text{ negative examples}}$$

We define the balanced accuracy as

$$\overline{A} \triangleq \frac{1}{2}(A_0 + A_1) \tag{2}$$

With these notations, we can verify that the accuracy is equal to $A = \frac{TP+TN}{TP+TN+FP+FN}$.
(ii) (4 point) Show that

$$\rho = \frac{TP + FN}{TP + TN + FP + FN}$$

and

$$A = \rho \cdot A_1 + (1 - \rho)A_0 \tag{3}$$

Comparing equation (2) and (3), we can see that the accuracy and balanced accuracy are both linear combination of A_0 and A_1 but with different weighting.
(iii) (3 point) Show that the trivial classifier you constructed for part (i) has balanced accuracy 50%.
Partly because of (iii), the balanced accuracy $\overline{A}$ is often a preferable evaluation metric than the accuracy A. Sometimes people also report the accuracies for the two classes (A_0 and A_1) to demonstrate the performance for each class. **Answer:**

i)

A trivial classifier can either guess always positive or negative. Regardless of the dataset, there exists a classifier with either $A = p$ or $A = 1 - p$.
For both cases, from a classifier with $p \leq 1 - p$ or $p > 1 - p$, we can say that our accuracy could be at least $1 - p$.

For $p > 1 - p$:

A trivial classifier with either have $A = p$ or $A = 1 - p$. Due to $p > 1 - p$, we can say that $A = p$ or $A = 1 - p$ which satisfies $A \geq 1 - p$

For $1 - p > p$:

A trivial classifier once again will either have $A = p$ or $A = 1 - p$.
Hence due to $1 - p \geq p$ we have:
i) $A = p \rightarrow A < 1 - p \rightarrow$ A always positive classifier is not fulfilling
ii) $A = 1 - p \rightarrow A > 1 - p \rightarrow$ Satisfies $A \geq 1 - p$.
$\Rightarrow$ For both cases, where $p > 1 - p$ or $p <= 1 - p$ there exists one classifier that satisfies $A \geq 1 - p$.

ii)

The amount of all examples is the sum of positive as well as negative examples #examples = TP+FN+TN+FP
The amount of only positive examples is #positive examples = TP+FN.
p is defined as the fraction of positive examples. Hence, it follows:

$$p = \frac{\#\text{positive examples}}{\#\text{examples}} = \frac{TP + FN}{TP + FN + TN + FP}$$

From before, we know:
#examples = TP+FN+TN+FP And correct predictions examples are:
#correct predictions = TP+TN
So, it follows:

$$A = \frac{TP + TN}{TP + FN + TN + FP}$$

$$A = \frac{TP}{TP + FN + TN + FP} + = \frac{TN}{TP + FN + TN + FP}$$

Let $N = TP + FN + TN + FP$:

$$A = \frac{TP}{N} + \frac{TN}{N}$$

$$A = \underbrace{\frac{TP}{TP + FN}}_{A_1} \cdot \underbrace{\frac{TP + FN}{N}}_{p} + \underbrace{\frac{TN}{TN + FP}}_{A_2} \cdot \underbrace{\frac{TN + FP}{N}}_{(1-p)}$$
$$= A_1 p + A_2(1 - p)$$

iii) The trivial classifier that always predicts negative, will have $A = 1 - p$.
It's accuracy on positive examples A_1 is zero, as we always predict negative.
The same way our accuracy A_0 is always 1.
This gives us: $\bar{A} = \frac{1}{2}(1 + 0) = 0.5$

(b) [5 points] **Coding problem: vanilla logistic regression**

First, we use the vanilla logistic regression to learn an imbalanced dataset. For the rest of the question, we will use the dataset and starter code provided in the following files:

- `src/imbalanced/{train,validation}.csv`
- `src/imbalanced/imbalanced.py`

Each file contains n examples, one example $(x^{(i)}, y^{(i)})$ per row. x is two-dimensional, i.e., the i-th row contains columns $x_1^{(i)} \in \mathbb{R}$, $x_2^{(i)} \in \mathbb{R}$, and $y^{(i)} \in \{0, 1\}$. Let $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^n$ be our training dataset. $\mathcal{D}$ has ρn examples with label 1 and $(1 - \rho)n$ with label 0. In the dataset we constructed, $\rho = 1/11$.

You will train a linear classifier $h_\theta(x)$ with average empirical loss for logistical regression, where $h_\theta(x) = g(\theta^T x), g(z) = 1/(1 + e^{-z})$, similar to Problem 3:

$$J(\theta) = -\frac{1}{n} \sum_{i=1}^{n} \left(y^{(i)} \log(h_\theta(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)})) \right),$$

You are encouraged to use your code for problem 3, but you are also allowed to use any standard logistic regression library or package to optimize the objective above. After obtaining the classifier, compute the classifier's accuracy (A), balanced accuracy ($\overline{A}$), accuracies for the two classes (A_0, A_1) on the validation dataset, and report them in the writeup. You are expected to observe that the minority class (positive class) has significantly lower accuracy than the majority class.

Create a plot to visualize the validation set with x_1 on the horizontal axis and x_2 on the vertical axis. Use different symbols for examples $x^{(i)}$ with true label $y^{(i)} = 1$ than those

with $y^{(i)} = 0$. On the same figure, plot the decision boundary obtained by your model (i.e, line corresponding to model's predicted probability = 0.5) in red color. Include this plot in your writeup.

Answer:

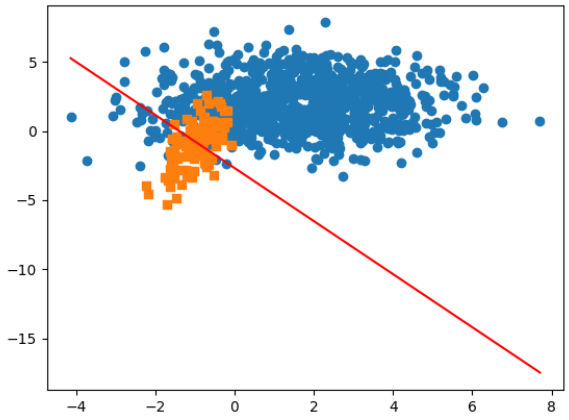


Figure 6: Vanilla Logistic Regression on imbalanced set

The classifiers accuracy values are the following:

```
A1: 0.57, A0: 0.975, A: 0.9381818181818181, balanced A: 0.7725
```

This shows that the accuracy of positive examples is worse than the one of negative results

(c) [5 points] **Re-sampling/Re-weighting Logistic Regression**

The relatively low accuracy for the minority class and the resulting low balanced accuracy are undesirable for many applications. Various methods have been proposed to improve the accuracy for the minority class, and learning imbalanced datasets is an active open research direction. Here we introduce a simple and classical re-sampling/re-weighting technique that helps improve the balanced accuracy in most of the cases.

We observe that the logistic regression algorithm works well for the accuracy but not for the balanced accuracy. Let $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{n}$ denote the existing training dataset. We will create a new dataset $\mathcal{D}'$ such that the accuracy on $\mathcal{D}'$ is the same as the balanced accuracy on $\mathcal{D}$.

Assume $\rho < 1/2$ without loss of generality, and let $\kappa = \frac{\rho}{1-\rho}$. This means the number of positive examples is κ times the number of negative examples in $\mathcal{D}$. Assume for convenience $1/\kappa$ is an integer.[1] Let $\mathcal{D}'$ be the dataset that contain each negative example once and $1/\kappa$ repetitions of each positive example in $\mathcal{D}$. Then we will apply the logistic regression on the new dataset $\mathcal{D}'$.

Prove that for any classifier, the balanced accuracy on $\mathcal{D}$ is equal to the accuracy on $\mathcal{D}'$. Moreover, **show that** the average empirical loss for logistical regression on the dataset $\mathcal{D}'$ is equal to

[1] otherwise we can round up to the nearest integer with introducing a slight approximation.

$$J(\theta) = -\frac{1+\kappa}{2n}\sum_{i=1}^{n} w^{(i)}\left(y^{(i)}\log(h_\theta(x^{(i)})) + (1-y^{(i)})\log(1-h_\theta(x^{(i)}))\right),$$

where $w^{(i)} = 1$ if $y^{(i)} = 0$ and $w^{(i)} = 1/\kappa$ if $y^{(i)} = 1$.
Observe effectively we are re-weighting the loss function for each example by some constant that depends on the frequency of the class it belongs to.
Answer:

Given:

i. $p < 0.5 \rightarrow$ The minority class occurs with p
The majority class occurs with (1-p)
ii. $K = \frac{p}{1-p} \rightarrow$ The ratio between p & (1-p)

Prove $\bar{A_D} = A_D$:

$$A_D = \frac{1}{2}(A_0 + A_1)$$

The new dataset D' has as many negative examples N_0 as D but a new amount of positive examples $N_1' = N_1 \cdot \frac{1}{K}$
The accuracy of D' is as following:

$$A_D' = \frac{TP + TN}{TP + FN + TN + FP}$$

i. By definition, follows: $TP' = A_1(TP' + FN') = A_1 N_1'$
$TN' = A_0(TN' + FP') = A_0 N_0$

We get: $A_D' = \frac{A_1 N_1' + A_0 N_0}{N_1' + N_0}$
With $k = \frac{p}{1-p} \rightarrow \frac{N_1}{N_0}$, we can derive $N_1 = N_0 K$. With this we get $N_1' = N_0 k \frac{1}{k}$.

Inserting this into 3)

$$3) \quad A_D' = \frac{A_1 N_0 + K A_0 N_0}{N_0 + K N_0} = \frac{A_1 + A_0}{2} = \frac{1}{2}(A_1 + A_0)$$

Weighted loss function:

As in D' every positive example occurs $\frac{1}{K}$ as often, we need to weigh the training examples accordingly.

So, we can express define a weight $w^{(i)} = \begin{cases} 1 & \text{if } y^{(i)} = 0 \\ \frac{1}{K} & \text{if } y^{(i)} = 1 \end{cases}$ so that our data gets weighted accordingly.
With a total of $N' = N_0 + N_1' = 2N_0$, we can say:

$$J(\theta) = -\frac{1}{N'}\sum_{i=1}^{N'} w^{(i)}\left(y^{(i)}\log(h_\theta(x^{(i)})) + (1-y^{(i)})\log(1-h_\theta(x^{(i)}))\right)$$

With $N_0 = \frac{n}{(1+k)}$ we can say, $N' = \frac{2n}{(1+k)}$

$$J'(\theta) = -\frac{1+k}{2n} \sum_{i=1}^{2N_0} w^{(i)} \left(y^{(i)} \log(h_\theta(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)})) \right)$$

(d) [5 points] **Coding problem: re-weighting minority class**

In `src/imbalanced/imbalanced.py`, implement the logistic regression algorithm on $\mathcal{D}'$. Compute and report the classifier's accuracy (A), balanced accuracy ($\overline{A}$), accuracies for the two classes (A_0, A_1) on the validation dataset. You are expected to see that the accuracy of minority class (class 1) improved significantly whereas that of the majority class dropped compared to vanilla logistic regression. However, the balanced accuracy is significantly greater than that of the vanilla logistic regression.

Create a plot to visualize the validation set with x_1 on the horizontal axis and x_2 on the vertical axis. Use different symbols for examples $x^{(i)}$ with true label $y^{(i)} = 1$ than those with $y^{(i)} = 0$. On the same figure, plot the decision boundary obtained by your model (i.e, line corresponding to model's predicted probability = 0.5) in red color. Include this plot in your writeup.

Answer:

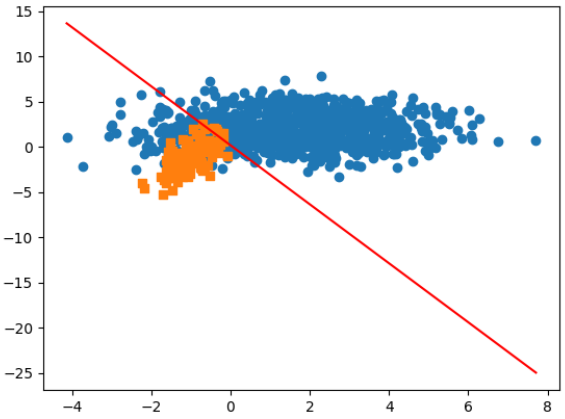


Figure 7: Reweighted Logistic Regression on imbalanced set

The classifiers accuracy values are the following:

```
A1: 0.94, A0: 0.885, A: 0.89, bal. A: 0.9125
```

This shows that the accuracy of positive examples has increased significantly, and with that the balanced accuracy as well!

6. **[25 points] A Simple Neural Network**

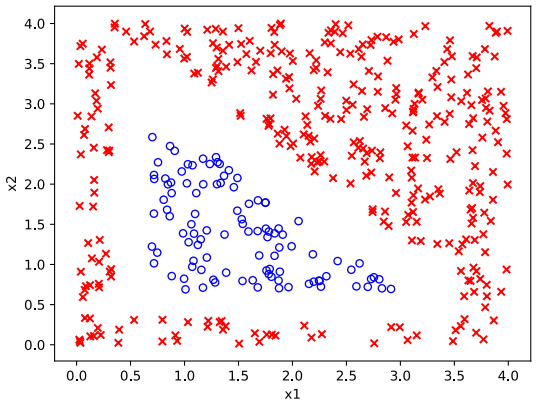


Figure 10: Plot of dataset X.

Let $X = \{x^{(1)}, \cdots, x^{(n)}\}$ be a dataset of n samples with 2 features, i.e $x^{(i)} \in \mathbb{R}^2$. The samples are classified into 2 categories with labels $y^{(i)} \in \{0, 1\}$. A scatter plot of the dataset is shown in Figure 10:

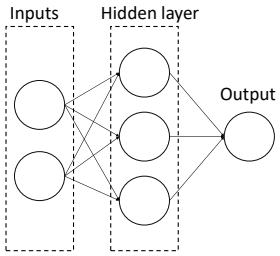


Figure 11: Architecture for our simple neural network.

The examples in class 1 are marked as as "×" and examples in class 0 are marked as "○". We want to perform binary classification using a simple neural network with the architecture shown in Figure 11:

Denote the two features x_1 and x_2, the three neurons in the hidden layer h_1, h_2, and h_3, and the output neuron as o. Let the weight from x_i to h_j be $w_{i,j}^{[1]}$ for $i \in \{1, 2\}, j \in \{1, 2, 3\}$, and the weight from h_j to o be $w_j^{[2]}$. Finally, denote the intercept weight for h_j as $w_{0,j}^{[1]}$, and the intercept weight for o as $w_0^{[2]}$. For the loss function, we'll use average squared loss instead of the usual negative log-likelihood:

$$l = \frac{1}{n} \sum_{i=1}^{n} \left(o^{(i)} - y^{(i)} \right)^2,$$

where $o^{(i)}$ is the result of the output neuron for example i.

(a) [5 points] Suppose we use the sigmoid function as the activation function for h_1, h_2, h_3 and o. What is the gradient descent update to $w_{1,2}^{[1]}$, assuming we use a learning rate of α? Your answer should be written in terms of $x^{(i)}$, $o^{(i)}$, $y^{(i)}$, and the weights.

Answer:

The GD update to $w_{1,2}^{[1]}$ should be the gradient of the loss w.r.t to $w_{1,2}^{[1]}$:

$$\frac{\partial L}{\partial w_{1,2}^{[1]}} = \frac{\partial L}{\partial o}\frac{\partial o}{\partial z_o}\frac{\partial z_o}{\partial h_2}\frac{\partial h_2}{\partial z_2}\frac{\partial z_2}{\partial w_{1,2}}$$

$$\frac{\partial l}{\partial o} = 2(o-y)$$
$$\frac{\partial o}{\partial z_o} = \sigma'(z_o) = o(1-o)$$
$$\frac{\partial z_o}{\partial h_2} = w_2$$
$$\frac{\partial h_2}{\partial z_2} = \sigma'(z_2) = h_2(1-h_2)$$
$$\frac{\partial z_2}{\partial w_{1,2}^{[1]}} = x_1$$

$\rightarrow$ The full GD update rule with learning Rate α would be:

$$W_{1,2}^{[1]} := w_{1,2}^{[1]} - \alpha\left[\frac{1}{n}\sum_{i=1}^{n} 2(o^{(i)} - y^{(i)}) \cdot o^{(i)}(1-o^{(i)}) \cdot w_2 \cdot h_2^{(i)}(1-h_2^{(i)}) \cdot x_1^{(i)}\right]$$

(b) [10 points] Now, suppose instead of using the sigmoid function for the activation function for h_1, h_2, h_3 and o, we instead used the step function $f(x)$, defined as

$$f(x) = \begin{cases} 1, x > 0 \\ 0, x \leq 0 \end{cases}$$

Is it possible to have a set of weights that allow the neural network to classify this dataset with 100% accuracy?

If you believe it's possible, please implement your approach by completing the `optimal_step_weights` method in `src/simple_nn/simple_nn.py` and including the corresponding `step_weights.pdf` plot showing perfect prediction in your writeup.

If it is not possible, please explain your reasoning in the writeup.

Hint 1: There are three sides to a triangle, and there are three neurons in the hidden layer.

Hint 2: A solution can be found where all weight and bias parameters take values only in $\{-1, -0.5, 0, 1, 3, 4\}$. You are free to come up with other solutions as well.

Answer: The step function introduces a non-linearity that, in principle, allows us to construct a triangular decision boundary. In this setup, each neuron is responsible for modeling one side of the triangle.

If we assume that each neuron's weights are orthogonal to the decision boundary it represents, then the dot product of the weights and the input determines on which side of the boundary the input lies. Applying the step function to this result yields a value of 1 or 0, indicating the side of the boundary.

(c) [10 points] Let the activation functions for h_1, h_2, h_3 be the linear function $f(x) = x$ and the activation function for o be the same step function as before.

Is it possible to have a set of weights that allow the neural network to classify this dataset with 100% accuracy?

If you believe it's possible, please implement your approach by completing the `optimal_linear_weights` method in `src/simple_nn/simple_nn.py` and including the corresponding `linear_weights.pdf` plot showing perfect prediction in your writeup.

If it is not possible, please explain your reasoning in the writeup.

Hint: The hints from the previous sub-question might or might not apply.

Answer: Using only linear functions as activation functions will not work. Even if the output activation function is a step function, we need to distinguish which side of each specific (linear) decision boundary—drawn by h_1, h_2, and h_3—the input lies on. However, in the output o, all these boundaries collapse into a single linear calculation, making it impossible to differentiate between the sides.

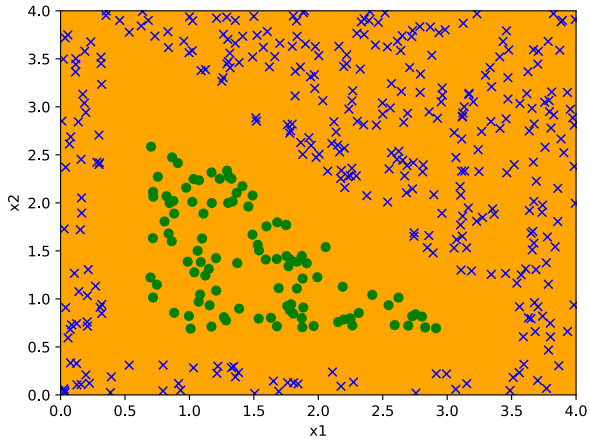


Figure 12: Decision Boundary – Linear Activations